

# Deep Learning for NLP

## Training LLMs

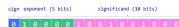
## Memory and compute requirements

32-bit float (FP32)



$$(-1)^0 \times 2^{100-1023} \times 1.5707964 = 3.1415927$$

16-bit float (FP16)



$$(-1)^0 \times 2^{100-1023} \times 1.571 = 3.141$$

## Learning goals

- Learn about different contributions to compute requirements
- Learn how model size components influence memory requirements

# FROM TRANSFORMER ARCHITECTURE TO TRAINING COST

You already know how a Transformer is built.

Now we want to turn its architecture into concrete numbers:

- **How many parameters does it have?**

- How do  $d_{\text{model}}$ , number of layers, and other architectural choices determine model size?

- **How much compute does training require?**

- How do model size and number of training tokens determine the number of operations?

- **How much GPU memory does training require?**

- Parameters are only one part: we also need to store gradients, optimizer states, and activations.

**Architecture**  $\longrightarrow$  **parameters**  $\longrightarrow$  **compute and memory**

# WHERE ARE THE PARAMETERS IN A TRANSFORMER BLOCK?

A decoder Transformer block mainly contains parameters in **self-attention** and the **feed-forward network (MLP)**.

## Self-attention:

- Query, key, and value projections:

$$W_Q, W_K, W_V \quad d_{\text{model}} \times (n_{\text{heads}} d_{\text{head}})$$

- Output projection:

$$W_O \quad (n_{\text{heads}} d_{\text{head}}) \times d_{\text{model}}$$

## Feed-forward network (MLP):

- Two weight matrices, typically with hidden dimension  $4d_{\text{model}}$ :

$$d_{\text{model}} \times 4d_{\text{model}} \quad \text{and} \quad 4d_{\text{model}} \times d_{\text{model}}$$

In the standard Transformer,

$$n_{\text{heads}} d_{\text{head}} = d_{\text{model}}.$$

# PARAMETER COUNT OF ONE TRANSFORMER BLOCK

Using  $n_{\text{heads}} d_{\text{head}} = d_{\text{model}}$ , self-attention ( $W_Q, W_K, W_V, W_O$ ) contributes

$$P_{\text{attn}} = 4d_{\text{model}}^2$$

The MLP contributes

$$P_{\text{MLP}} = 2 \cdot d_{\text{model}} \cdot 4d_{\text{model}} = 8d_{\text{model}}^2.$$

Hence,

$$P_{\text{block}} = 12d_{\text{model}}^2$$

and for  $n_{\text{layers}}$ ,

$$P_{\text{blocks}} = 12n_{\text{layers}}d_{\text{model}}^2$$

# DOES THE APPROXIMATION WORK?

**GPT-3 Small** ( $n_{\text{layers}} = 12$ ,  $d_{\text{model}} = 768$ ): ► Brown et al., 2020: GPT-3

$$P_{\text{blocks}} \approx 12 \cdot 12 \cdot 768^2 \approx 85\text{M}.$$

Actual model size: **125M parameters.**

**GPT-3 Medium** ( $n_{\text{layers}} = 24$ ,  $d_{\text{model}} = 1024$ ):

$$P_{\text{blocks}} \approx 12 \cdot 24 \cdot 1024^2 \approx 302\text{M}.$$

Actual model size: **350M parameters.**

**What is missing? Mainly the token embeddings.**

For vocabulary size  $|\mathcal{V}|$ ,

$$P_{\text{embed}} = |\mathcal{V}| d_{\text{model}}.$$

GPT-3 has  $|\mathcal{V}| \approx 50,000$ , hence approximately

$$P_{\text{embed}} \approx 39\text{M (Small)}, \quad 51\text{M (Medium)}.$$

Other parameters (e.g., LayerNorms and biases) are comparatively small.

# TRAINING COMPUTE: FORWARD PASS

Let  $P$  be the number of parameters and  $D$  the number of training tokens.

For one token, each parameter participates in roughly one multiply-add operation.

A multiply-add costs approximately 2 FLOPs:

$$a \cdot b + c$$

Therefore, over  $D$  training tokens,

$$C_{\text{forward}} \approx 2PD$$

Hence, forward-pass compute grows linearly with

- the number of model parameters  $P$ ,
- the number of training tokens  $D$ .

# TRAINING COMPUTE: BACKWARD PASS

In the forward pass, a layer computes

$$Y = XW.$$

During backpropagation, it must compute

- gradients for the weights  $W$ ,
- gradients for the layer input  $X$  (to continue backpropagation).

Both require matrix multiplications of similar size to the forward pass.

Thus, backward costs roughly twice as much as forward:

$$C_{\text{backward}} \approx 4PD.$$

Together,

$$C \approx 2PD + 4PD = 6PD$$

# FROM FLOPS TO TRAINING TIME

Training requires  $C \approx 6PD$  floating-point operations.  
If the hardware achieves throughput  $\tau$  FLOP/s, then

$$T \approx \frac{C}{\tau} = \frac{6PD}{\tau}$$

► Source: Quentin et al., 2023

**Example:** 7B parameters trained on 1T tokens:

$$C \approx 6 \cdot 7 \cdot 10^9 \cdot 10^{12} = 4.2 \cdot 10^{22} \text{ FLOPs.}$$

Training can be distributed across multiple GPUs. Ideally,

$$\tau \approx n_{\text{GPU}} \cdot \tau_{\text{GPU}}.$$

In practice, communication and synchronization prevent ideal linear scaling.

# A FIXED COMPUTE BUDGET LEAVES A CHOICE

Training compute is approximately

$$C \approx 6PD,$$

where

- $P$  = number of model parameters,
- $D$  = number of training tokens.

For a fixed compute budget  $C$ ,

$$PD \approx \text{constant}.$$

So we can choose between, for example,

- a **larger model** trained on fewer tokens,
- a **smaller model** trained on more tokens.

**Which choice gives the best model?**

This question (and similar tradeoffs) is addressed by **scaling laws**.

# WHAT OCCUPIES GPU MEMORY?

**How big will the model be in bytes? Will it fit on my GPU?**

Memory is approximately

number of stored values  $\times$  bytes per value.

For **inference**, we mainly store

- model parameters,
- activations.

Training additionally requires

- gradients,
- optimizer states.

Thus,  $M_{\text{train}} = M_{\text{params}} + M_{\text{grad}} + M_{\text{opt}} + M_{\text{activ}}$

$M_{\text{params}}$ ,  $M_{\text{grad}}$ , and  $M_{\text{opt}}$  scale with the number of parameters.

Activation memory  $M_{\text{activ}}$  also depends on sequence length and batch size.

# FLOATING-POINT PRECISION

Floating-point numbers use bits for

- the sign,
- the exponent,
- the significand.

Two common 16-bit formats are:

|      | sign | exponent | significand |
|------|------|----------|-------------|
| FP16 | 1    | 5        | 10          |
| BF16 | 1    | 8        | 7           |

BF16 uses more bits for the exponent and fewer for the significand:

- larger dynamic range,
- lower numerical precision.

Both require 2 bytes/value.

For comparison, FP32 requires 4 bytes/value.

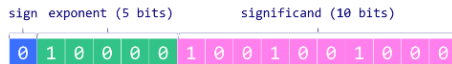
# FP32/FP16 ( ▶ (MAXIME LABONNE) )

## 32-bit float (FP32)



$$(-1)^0 \times 2^{128-127} \times 1.5707964 = 3.1415927$$

## 16-bit float (FP16)



$$(-1)^0 \times 2^{128-127} \times 1.571 = 3.141$$

You can represent every float like this:  $(-1)^S \cdot 2^{e-bias} \cdot 1.m$

(bias = 15 for FP16, 127 for BF16/FP32)

Modern accelerators also support FP8 and even FP4 for selected training operations; mixed precision remains an active area of research.

# TRAINING MEMORY PER PARAMETER

Assume BF16 parameters and gradients, and AdamW.  
For each parameter, training stores:

| Quantity           | Memory  |
|--------------------|---------|
| BF16 parameter     | 2 bytes |
| BF16 gradient      | 2 bytes |
| FP32 Adam momentum | 4 bytes |
| FP32 Adam variance | 4 bytes |

For a model with  $P$  parameters:

$$M_{\text{params+grad+opt}} \approx 12P \text{ bytes}$$

**Example:** for a 7B-parameter model,

$$12 \cdot 7 \cdot 10^9 = 84 \text{ GB.}$$

This does **not** yet include activation memory.

# WILL MY MODEL FIT ON A GPU?

Consider a 7B-parameter model.

BF16 parameters:  $2 \cdot 7 \cdot 10^9 = 14$  GB

Training state:  $12 \cdot 7 \cdot 10^9 = 84$  GB

Activations add further memory and depend strongly on batch size, sequence length, and architecture.

► Lv et al., ACL 2024 report about **31% activation memory** for one LLaMA-7B setup.

Activation memory can often be traded for extra *re-computation* (*activation checkpointing*):

|                                 |
|---------------------------------|
| less memory $\iff$ more compute |
|---------------------------------|